

SEHH3140

PROGRAMMING PROJECT

Architectural Design Report

Group: Individual Group (Separated from Group 8)

Student ID: 24121627A

Student Name: YU Shing Hei

Contents

Introduction.....	3
Project Vision.....	3
Background	5
Specification (System Analyse)	6
Reality Situation.....	6
User Requirement	7
Use Cases and Scenarios.....	8
Use Case Diagram.....	9
Data Flow Diagram.....	10
System Requirement	11
Functional Requirement.....	11
Non-functional Requirement (NFR)	13
System Modeling (System Design)	15
User Interface Design	15
The First Page – Index	16
The Branch Controller Page.....	18
The Authentication Page	20
Walkie-Typie Pages and Broadcast Pages	20
The Mod System.....	22
Entity Relationship Diagram (ERD).....	23
IndexedDB ERD	23
PostgreSQL ERD	24
Test Plan.....	25
System Test	25
Component Test (Unit/Integration Test)	26
Acceptance Test	26
System Architecture	27
List of Architecture of System	27
Architectural Diagram	28
Cost Estimation.....	29
Conclusion	29
Reference	30

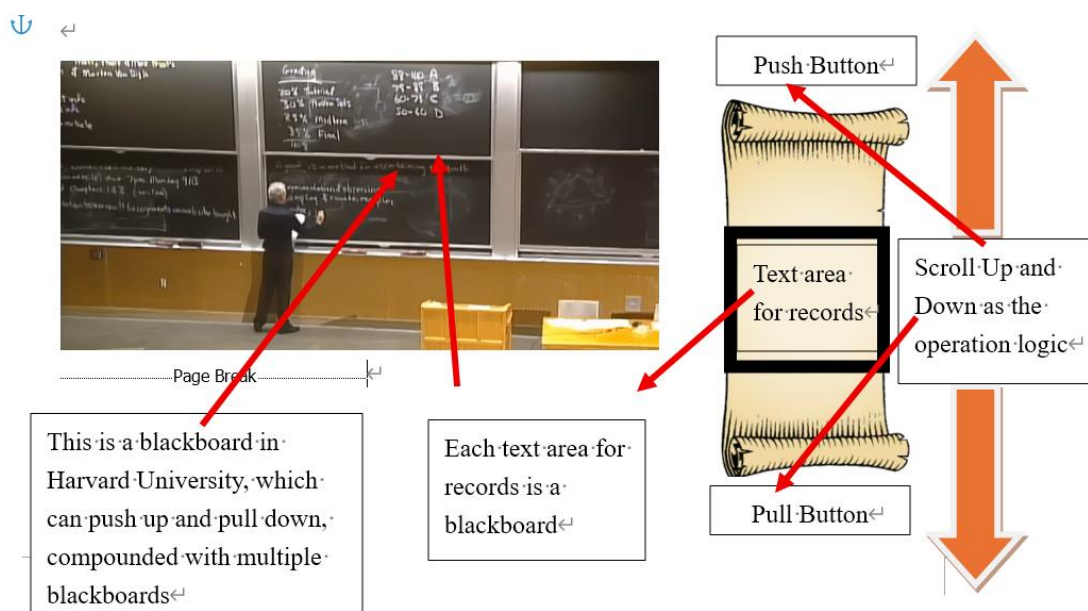
Introduction

The project we are building is a moddable notebook named "MyCLI". It stands for Clean Logging Interface, envisioning a minimalistic style notebook system with comprehensive fundamental features and highly customizable configurations, such that the website can be used in different ways based on the demands of users.

Project Vision

MyCLI adopts the sandbox philosophy that has been proven by video games like Minecraft and Rimworld. They are famous sandbox games that have no explicit goal. Instead, the goal is defined by users. Therefore, MyCLI is more like a framework which built-in with fundamental operations of text. We divided text-oriented operations into three parts: self-texting (Blackboard), one-to-one texting (Walkie-Typie), and one-to-many texting (Broadcast). Each of the namings indicates the concept:

1. **Blackboard:** This is the fundamental concept for the remaining two. Imagine an actual blackboard where lecturers used to highlight key information; this is a "page" of Blackboard. Next, imagine there exists a long scroll, where users can only read a part of it. When users need to read other parts, they need to push up and pull down; this is the concept of the PUSH and PULL concept for operating a Blackboard



2. **Walkie-Typie:** The source of the concept is obvious - the walkie-talkie, where users can instantly talk. We want to adopt this concept to text, where the typed text is streamingly shown on the other side.



3. **Broadcast:** There are two sources for this concept - the radio broadcast and the notice board. We want a place where users should actively visit and have a broadcast-wide propagation characteristic. This should be suitable for organization and the product changelog.



Moreover, we should introduce the modding system of our product. It is like the extension features from the browser, that every user can customize which mod to be activated to provide a unique user experience for each user. Some of the mods are merely theme changing or configurations, but some connect third-party API services, and even a client LLM that runs on their machine offline, which allows users to analyse the notebook by custom prompts.

In summary, we have board-based adoption of self-messaging, one-to-one messaging, and one-to-many messaging. Some might ask: Where is many-to-many?

Unfortunately, there already exists many brilliant software that are proficient in this domain, such as Discord. Adopting this feature would trade off the flexibility, which is obvious, as has been proven in video games: Most of the multiplayer games have weak compatibility with flexibility and mod culture.

Background

This project was inspired by video game culture, git and the command line interface; therefore, we referenced many terminologies from them. To avoid ambiguity, please do refer to the background section below for context:

1. **Clean "Logging" Interface:** The word "log" has different meanings in different areas. In computing, it refers to those automated record systems with immutable timestamps which serve engineers; in entertainment, log refers to intentional narrative artifacts created by fictional characters that serve the story. In this project, we are trying to combine both concepts together, utilizing the timestamp concept from log on computing, with less restriction, such that users can manipulate records easily.
2. **Git Concepts (Branch, Fork, Push, Pull, Commit, Checkout, Switch):** Git, which is a version control system (VCS) that offers powerful functions for developers to easily track, manage, and collaborate on changes made to the source code over time. In general, every developer holds a local copy of the repository. In this project, we are not going to duplicate a git VCS. Instead, we are using the local-first concept for this project with a branch management system for categorizing notes, an extra sectioning system based on timestamp inside of a branch, and capability for users to push and pull records via the server. Further terminology explanations will be mentioned on the corresponding sections in this report.
3. **Mods:** In video games' domain, mod indicates "modification", refers to alterations made to a video game that allow users to customize their game experience from visual retexturing to functional features. Initially, the UI/UX of this project prototype is considered non-user-friendly; therefore, we decided to append a mod controlling system that allows users to customize UX by activating the official-offered mods to change text expression, color theme, or even additional LLM features for aiding the UX. Moreover, due to the framework that offers various APIs for mod development, enabling programmers to develop their own one with greater ease.

Specification (System Analyse)

Reality Situation

There is literally zero shortage of text-oriented software available on the market today. However, every mainstream solution is opinionated system, where the developers decide the purpose, decide how it should be used, and users must conform.

Instant Messaging Apps as a Notebook to Self

WhatsApp is the most popular and accessible text tool in Hong Kong, but many of us repurpose the “message yourself” feature for agile note jotting and cross-device file transfer use, which is never the major design purpose. Therefore, it is comfortable, though, the “message yourself” is often stacked with untidied messages, impossible to categorize. Moreover, all the data is server-dependent, the interface is immutable, and registration requires a phone number, which is inflexible.

Knowledge Tools as Rigid Framework

The other representative spectrum of text-oriented software is Notion, which offers deep note management with databases, backlinks, and rich formatting. However, it trades off with complexity: each tool imposes a specific mental model, and each knowledge tool has different logic, such as Notion’s blocks, Obsidian’s Markdown vault, etc. Therefore, they are obviously too heavy to have less correlation for agile note jotting.

Community Platforms are Overkill

Discord is the most popular community platform, but it is somehow overkill for simple announcements for small organizations. Some people said they joined Discord just to browse the news from official, and they never chat.

Common Thread: Rigidity

Across all categories, they share the same limitation – they are all closed, finished products. All the features are fixed at release. The user cannot add translation in Discord, cannot add AI assistance to WhatsApp, and cannot reshape the interface that users personally like. The only way out is to switch to another equally rigid tool.

User Requirement

Suppose our project is capable of serving most kinds of people for different purposes, let's define the expectations. Users are supposed to be able to:

1. start recording at the first step on the website
2. view past records chronologically
3. organize categorized records
4. record at anytime with any devices
5. access same pieces of records from any devices
6. exchange data with another in real time
7. browse and read the latest published content (website changelog, operation guideline, etc.)
8. customize aids (translation, AI, etc.)

Use Cases and Scenarios

Let us mockup scenarios for different purposes:

Note: A fully uppercased word indicates a button.

Agile Jotting with LLM Aid (Covers Requirement 1, 2, 8):

Actor	Non-registered user (student)
Before-Condition	Website is accessible via browser; no login requirement
Behaviour Flow	1. Open website → Text area standing by (first page) 2. Enter records on the first page 3. PUSH to create new page; PULL to review history (editable) 4. Customize configurations as users' intention (e.g. max history slots) 5. Activate LLM mod from mod page → custom prompt querying
After-Condition	1. Records persist in IndexedDB 2. Auto clean old records by max history slots setting 3. LLM answers using records as context

Organizing with Branches (Covers Requirement 3):

Actor	Registered user (student)
Before-Condition	User has accumulated records in default branch "master"
Behaviour Flow	1. Create a new branch named "study" 2. SWITCH to branch "study" 3. Categorize and attach subject files on each page using PUSH and PULL 4. SWITCH between branches on demands 5. DELETE the branch when no longer needed
After-Condition	1. Records are categorized and each branches are manageable

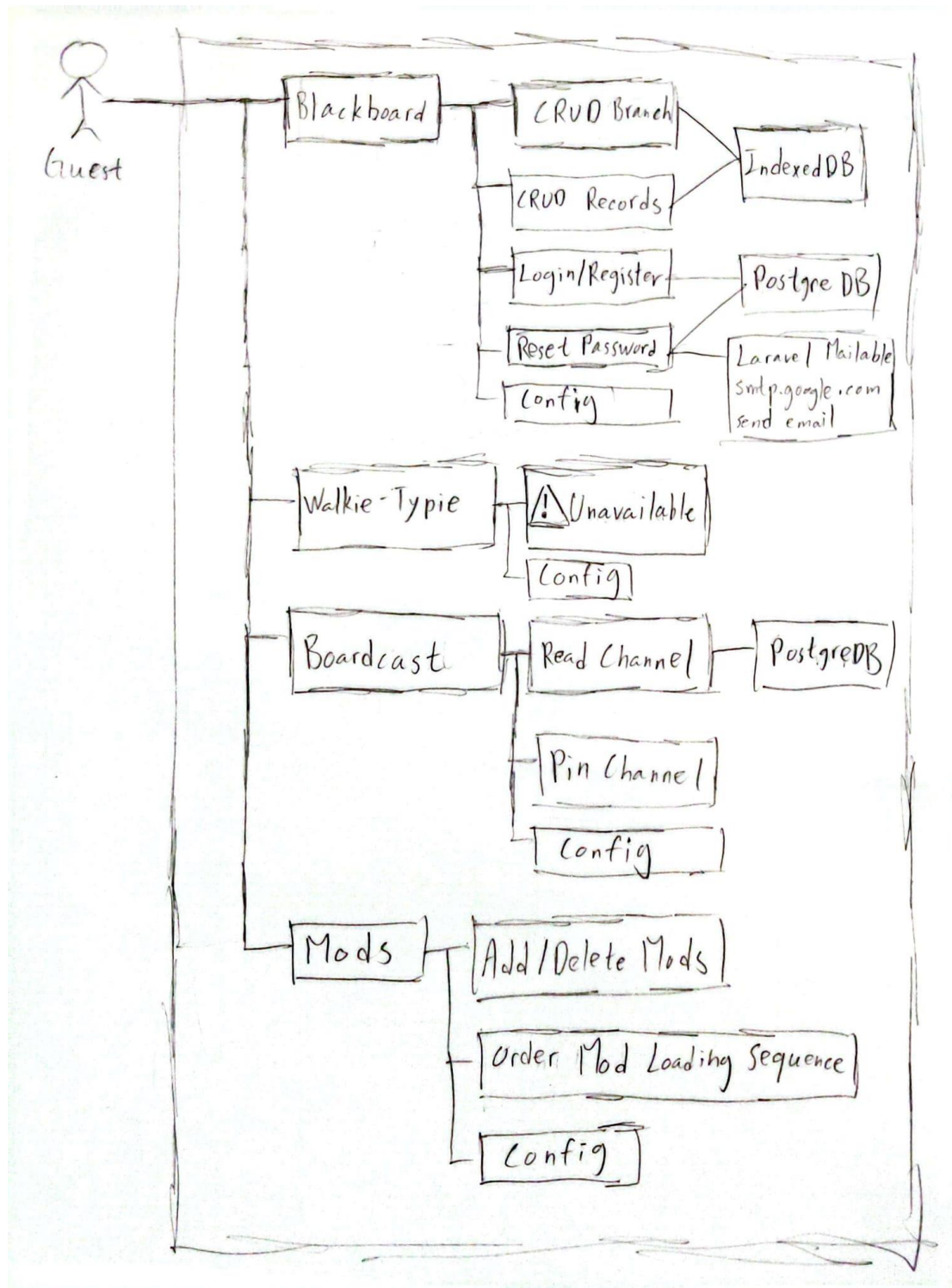
Data Transmission (Covers Requirement 4, 5, 6):

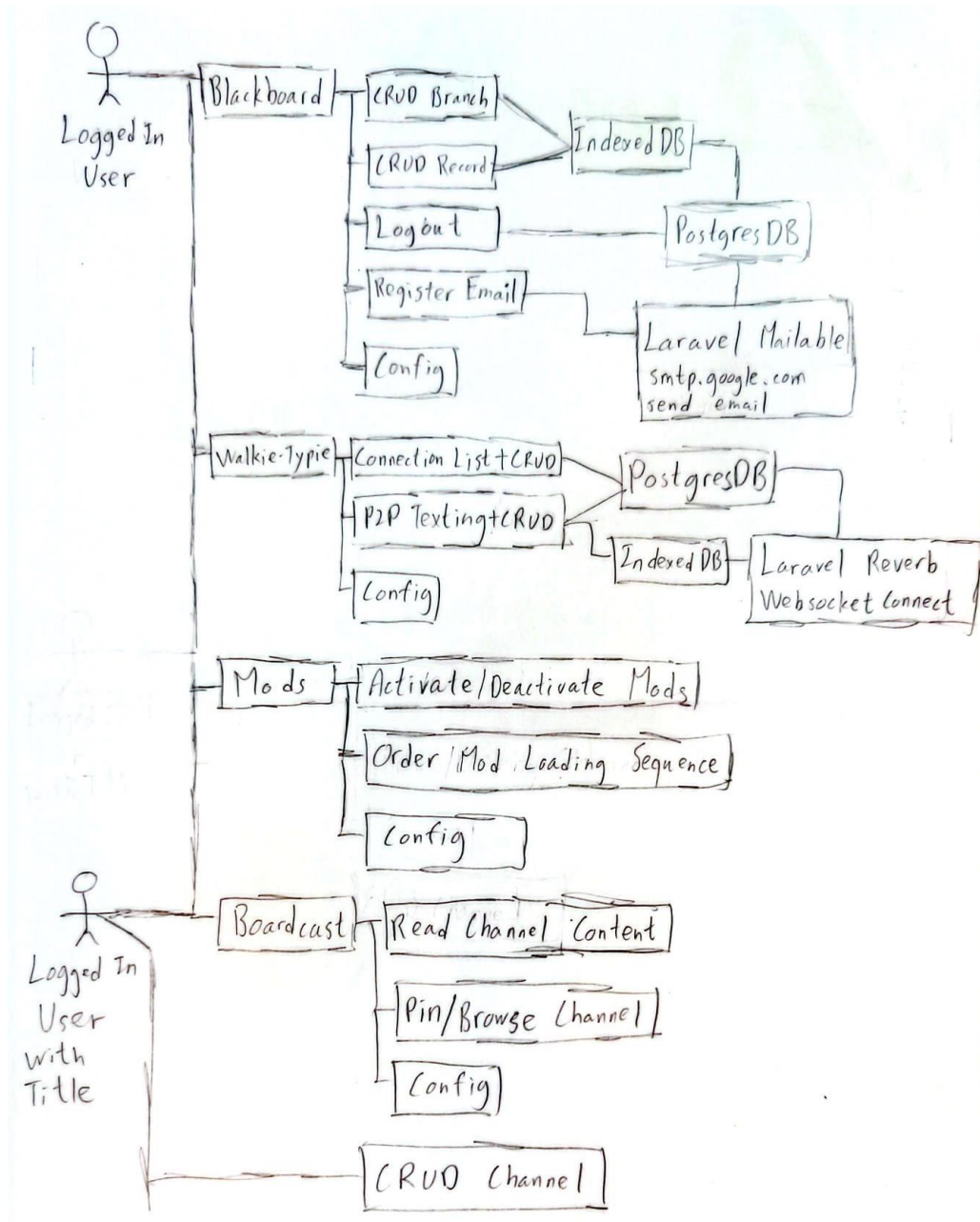
Actor	Two registered users (two students)
Before-Condition	User has accumulated records in default branch "master"
Behaviour Flow	Way 1: 1. Device A COMMIT to upload branch to server 2. Device B CHECKOUT to download branch data Way 2: 1. User A ADD User B by UID in Walkie-Typie 2. Real-time keystroke streaming via WebSocket (files are accepted)
After-Condition	1. Data is synced and persisted in all devices locally

Official Announcements (Covers Requirement 7):

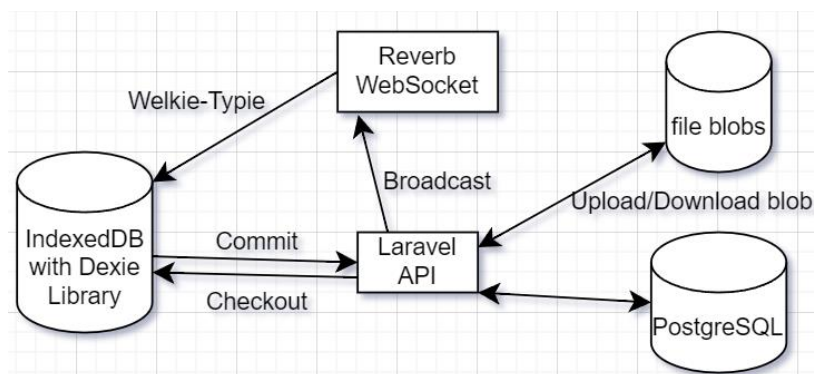
Actor	Titled user (lecturer) and untitled users (students)
Before-Condition	1. Titled UID is granted via backend (pgAdmin) 2. Server is hosted by organization
Behaviour Flow	1. Titled user CREATE a Broadcast channel; this channel follows the title of the author 2. Edit channel content using same operation logic as Blackboard 3. CAST the channel after edition 4. All users (including guests) can browse and read all channels 5. Only the title holder can edit the channel with the same title
After-Condition	1. Public announcements are accessible to all users

Use Case Diagram





Data Flow Diagram



System Requirement

Functional Requirement

Authentication

The system shall:

1. support registration/login/logout with only UID and password
2. support email binding after logging in
3. support reset password before logging in if the account has an email bound

Blackboard

The system shall:

1. provide immediately available text area on the first step of accessing the website
2. support file attachment on each page
3. store all records in IndexedDB to uphold the local-first principle
4. support simplistic chronological navigation: PUSH and PULL buttons to navigate records
5. support blank page auto-clean to improve the UX of navigation
6. support record auto-clean whenever accumulated records reach the dedicated threshold
7. order pages based on the timestamp of each page
8. support update timestamp on a page is updated, such that it goes to the front (assume user editing a page indicates the higher priority)
9. create a blank page after hitting the PUSH button on the latest record, such that the user can incrementally append records
10. support forking a branch, which creates a duplicate containing all previous records
11. support clearing a branch, which clears all records inside a branch
12. support CRUD on a branch
13. support branch renaming
14. support switching the branch from local
15. support upload/ download branch from server
16. support deleting the branch on the server
17. support configurations for the NO. 4, 5, 6 functional requirements

Walkie-Typie

The system shall:

18. support real-time peer-to-peer text communication (including file attachment) between two registered users
19. support connecting to each other by UID
20. support disconnect others
21. create a twin board for both sides after the connection
22. display both sides of the board on a single page
23. provide same operation logic as Blackboard does
24. auto-commit one side's board after editions
25. auto sync the other side's board
26. restrict the behaviour on not owned board (read only, still can PUSH and PULL pages)
27. support renaming connections
28. support configurations as Blackboard does

Broadcast

The system shall:

29. allow any users, including non-authenticated guests, to browse and read existing Broadcast channels
30. restrict behaviour for non-titled users on a channel (same logic as not owned board on Walkie-Typie)
31. provide the same logic of operation for the titled UID as the Blackboard page, and the branch does
32. restrict each channel's CRUD permission so that each channel created by a UID with a title, that channel is bound to the title of the creator; only the UID with the corresponding title can modify it

Mods

The system shall:

33. provide a series of officially made mods
34. support configuration for each mod
35. support instantiating a mod multiple times; each mod instance doesn't share configurations
36. support adding a mod functional button on a dedicated page, based on the definitions that are defined in each mod; each instance's button is independent

Non-functional Requirement (NFR)

Performance

The system shall:

1. auto save all board page content from Blackboard, Walkie-Typie, and Broadcast after an input action + 200ms debounce, to avoid too many system writing behaviours.
2. compress the HTTP responses to reduce bandwidth consumption; using Nginx Gzip, with a minimum length of 256 bytes for files
3. cache the frequently requested data to reduce server database burden; setting up Time To Live (TTL) for branch list, branch details, broadcast channels, with TTL smaller than 120 seconds
4. do SHA-256 for the attached files to deduplication, because the allowed maximum file size to be attached is 1GB. We do not expect multiple attachments of a 1GB file to create multiple instances; deduplicating by SHA-256 hashing could reduce the occupied storage of both client and server.
5. pre-cache the static resources, such that the website can be instantly loaded; we can use Service Worker to cache the HTML, CSS, JS, and audio files.
6. define the API request timeouts in 15s as the default, to avoid hanging connections.
7. use database indexing to optimize query performance

Reliability

The system shall:

8. eternalize the board data to clients' local before any sync behaviour to the server
9. auto restart the server services (Docker Containers)
10. retry the queue jobs after it fails for 3 times
11. maintain the server database referential integrity via cascading deletes
12. auto clean orphaned files (not being attached by any of the pages) after 24 hours to prevent residuals
13. provide structured error responses for all API failures, such as 400, 404, and 401 errors.
14. use SHA-256 hashing for content-addressed storage for integrity verification and deduplication, as mentioned in the performance NFR.
15. update the indexedDB version on data structure changes to prevent data loss.

Usability

The system shall:

16. officially support fundamental theme-changing extensions for language and color patterns, such as Chinese and Light Mode.
17. provide i18n json files for anyone to easily customize their own UI texts
18. provide responsive layout across mobile and desktop
19. support touch-based interactions
20. installable as a Progressive Web App (PWA) for an offline use case
21. provide contextual hints for complicated features
22. provide audio feedback for navigation actions
23. animate transitions
24. provide rich customizable options of the NFR of NO. 16, 19, 20, 21, 22, etc.
25. provide creative officially made mods as templates, also some API for mods provided as the framework, such that other developers can more easily develop their own mods

Security

The system shall:

26. hash the passwords before any storage behaviour
27. implement rate limiting for API ports to prevent abusement
28. validate the authentication input on the client, such as password, email, and UID length and format.
29. validate the attached files, reject high-risk file extensions such as php, exe, and html, etc.
30. provide safer DOM operations; never use *innerHTML*, but *textContent*, to avoid Cross-Site Scripting (XSS)
31. parameterize queries to prevent SQL injections
32. set security headers to mitigate common web attacks
33. protect sessions with secure cookie attributes
34. use content-addressed hashing (SHA-256) as an access token for file accesses
35. isolate backend services from direct external access
36. defined a *.gitignore* file for the GitHub repository of this project
37. define a *.env* file that includes all API keys and passwords, such as smtp.google.com and *pgAdmin* configurations; the *.env* file will be included in *.gitignore*, we will provide a *.env.example* file instead
38. define *.htaccess* to reject unpermitted access requests (Obsoleted, because the server app is changed from Apache to Docker + Nginx)

Portability

The system shall:

1. auto adapt the most suitable CSS depending on the screen size, operating system, and the browser
2. provide a comprehensive Dockerized system and push it to GitHub, so that other developers can be easier to one-shot deploy and host the server
3. provide .env.example with defined variable names for easier deployment for developers

System Modeling (System Design)

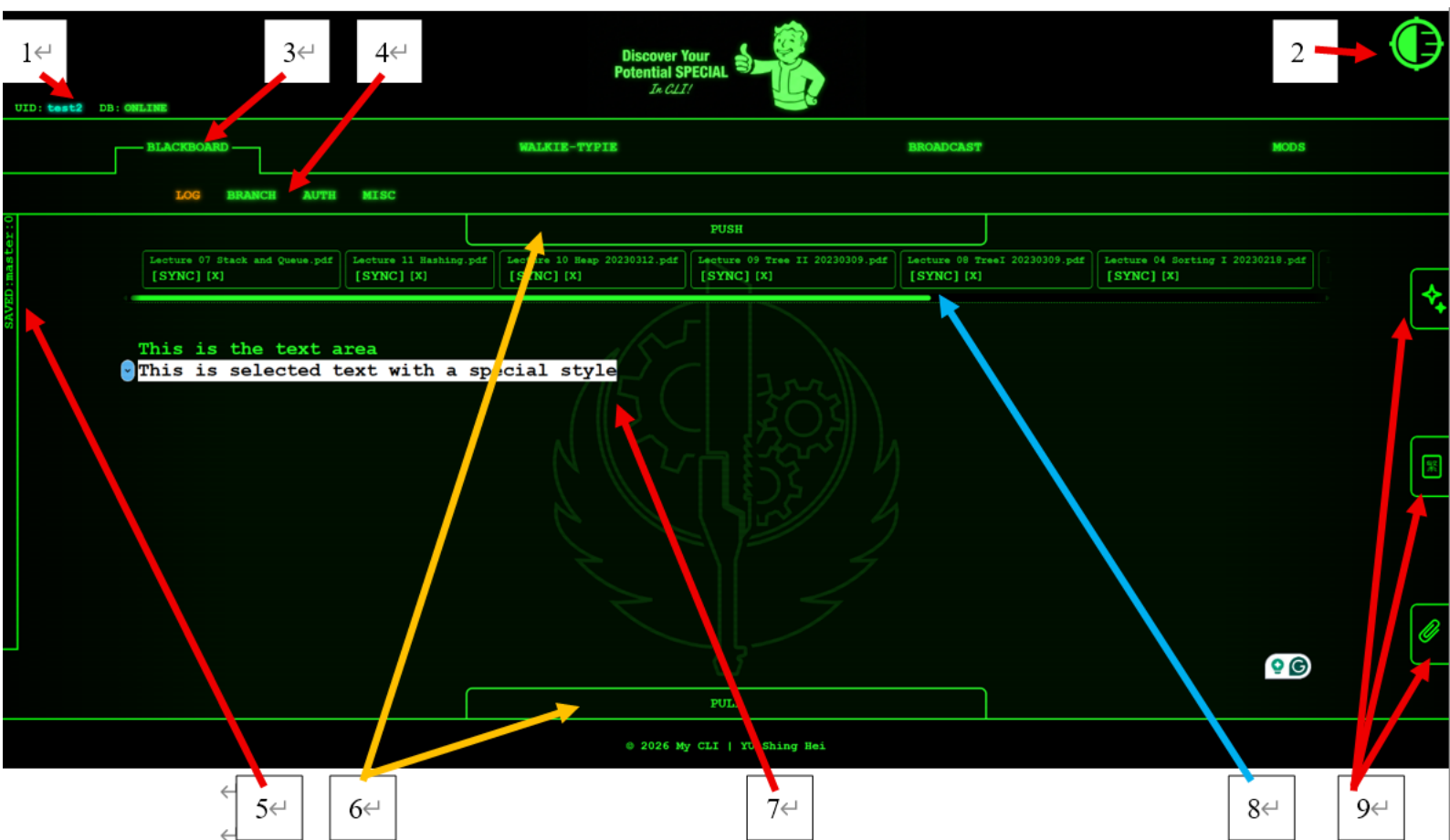
This section will translate the functional and non-functional requirements defined above into concrete design models. This will cover both server side (Laravel and PostgreSQL, etc.) and the client side (vanilla HTML, CSS, JavaScript, Dexie IndexedDB , etc.), reflecting MyCLI's local-first architecture.

User Interface Design

We have referenced the UI/UX design from one of the famous games for the initial design of the video game – Pip-Boy in Fallout 4.



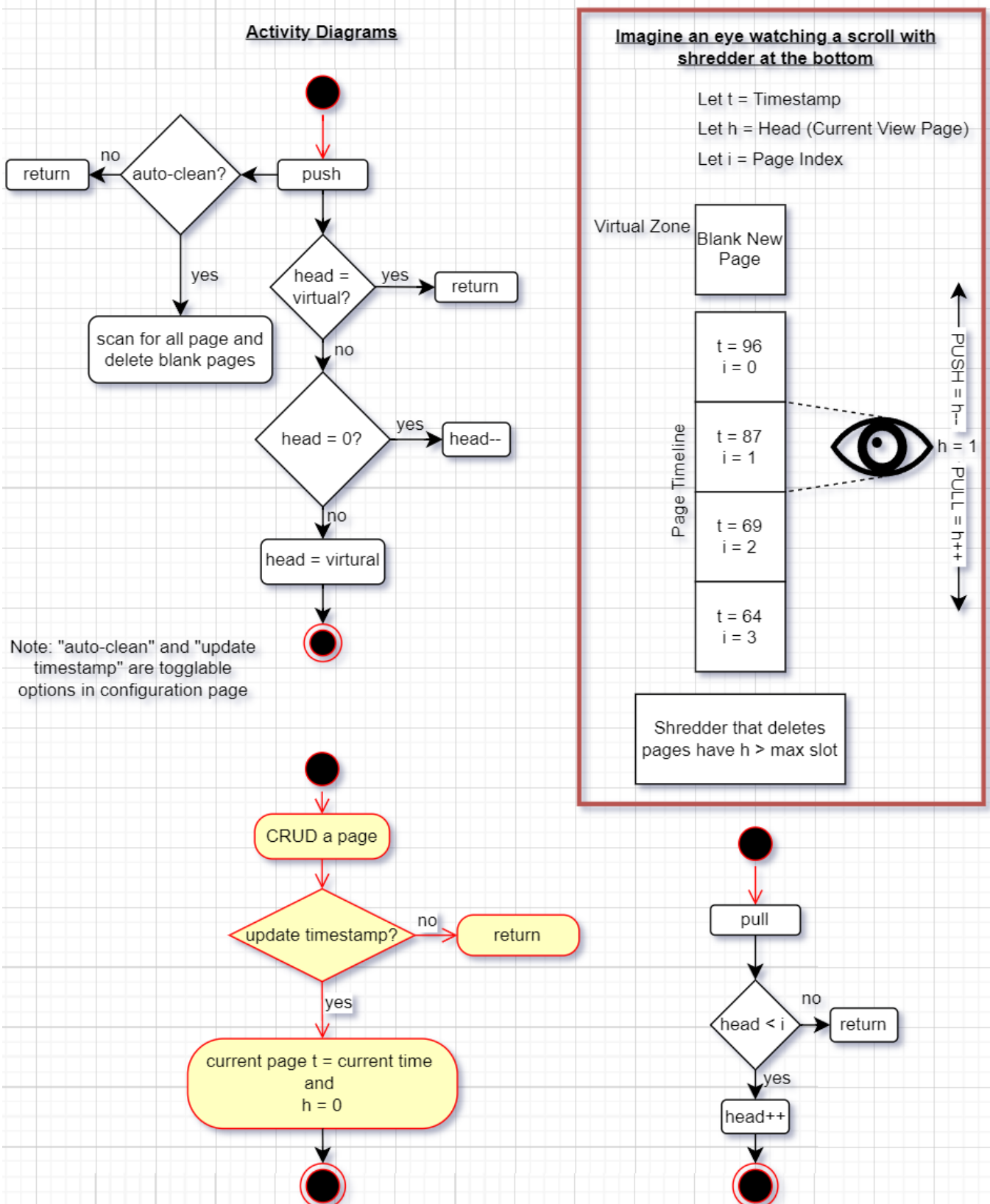
The First Page – Index



- 1: Head Up Display (HUD), which displays if the client is connected to the server, and the login status
- 2: The theme color pattern switching button
- 3: The navigation feature has four items in total. This feature only has click interaction.
- 4: The sub-navigation, each navigation item has its own sub-navigation item. This feature cannot only be interacted with by clicking, but it can also be hovered over and scrolled by wheel, or swaped on mobile devices.
- 5: The branch status indicator, which shows which branch you are located in and which page of a branch you are located in.
- 6: The page navigator that changes the page that the user is looking at.
- 7: The text area of a page
- 8: The file attachment chip. All the attached files are listed here, support scroll behaviour to view overflowed files
- 9: The mod shelf. All activated mods that have the shelf feature are shown here.

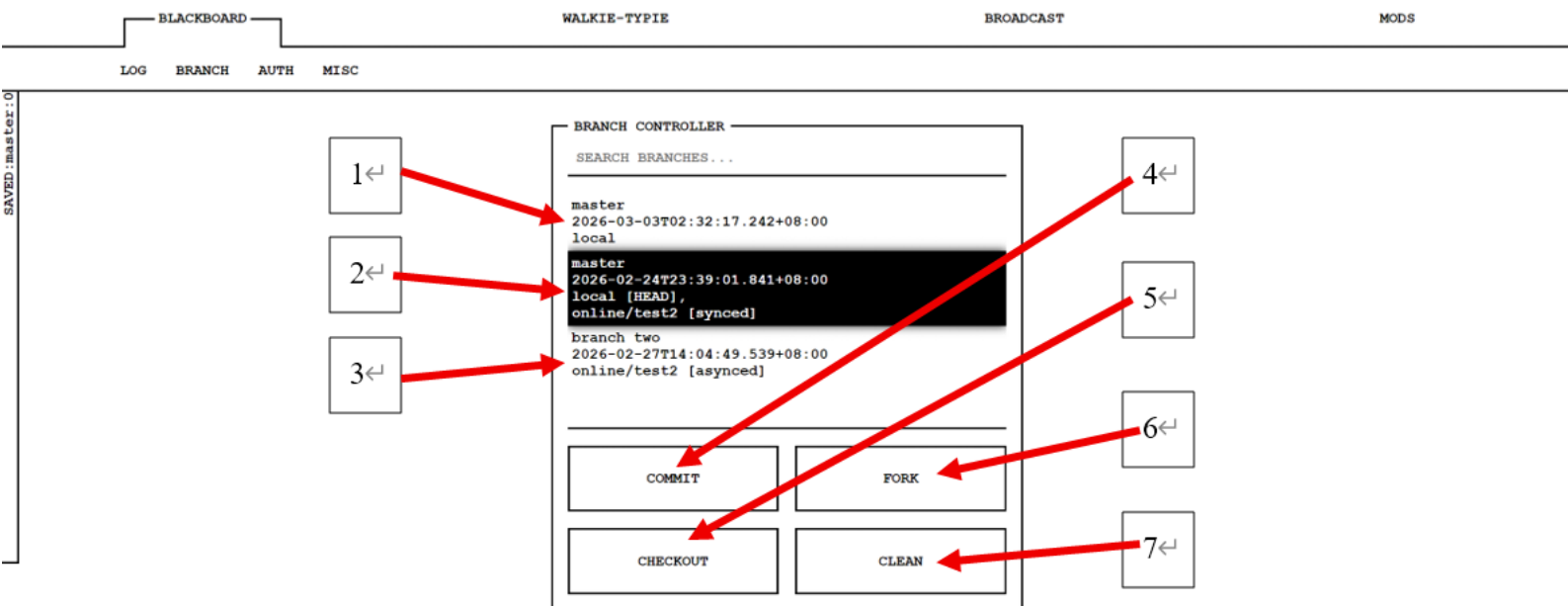
A Board Operation Logic

All components (Blackboard, Walkie-Typie, Broadcast) share the same fundamental concept – board. Here are the high-level diagrams and an analogy for users to understand the operation logic of push, pull, and head more easily.



The Branch Controller Page

Note: The developer of MyCLI loves the design using the CRT green color pattern. However, all of the testers responded with negative feedback. Therefore, we will use the light theme for further demonstration of UI design, which is a mod feature, in case you don't accept this either.



First, there are in total three kinds of status of a branch, in three combinations of indicators:

1. “local” (Local Only): That indicates the branch data is on clients’ local machine, not on the server
2. “local, online/uid” (Local and Server): That indicates the branch is on both clients’ local machine and on the server
3. “online/uid” (Server Only): That indicates the branch is on the server but not on the client

Additionally, there are indicators for the above status:

- [HEAD]: This indicates the branch where user is located at
- [syncd]: This indicates the branch data and the server data are the same
- [asynced]: This indicates the branch data doesn’t match the one on the server

Secondly, there are four operation buttons:

4. COMMIT: It means upload the selected branch to the server
5. CHECKOUT: This is a **polymorphic button** (Explain on the next page)
6. FORK: It means duplicating/cloning the selected branch
7. CLEAN: It is a **polymorphic button** (Explain on the next page)

```

master
2026-02-24T23:39:01.841+08:00
local [HEAD],
online/test2 [syncd]

```

This is a branch that has all status. Let us break them down line by line:

The first line “master” is the nickname that can be changed by users.

The second line is the immutable timestamp that is created on the moment when a branch is created. This is the primary key of a branch.

The third line is the local status indicator, if the user is positioned on this branch, then [HEAD] is indicated; if the branch is not on the local storage, then this line will not appear.

The fourth line is the online status indicator, the “test2” is the UID - the owner of this branch.

Polymorphic Branch Operation Buttons

The Branch Controller had two polymorphic buttons as mentioned last page. Each button dynamically changes the label and behaviour based on the state of the selected branch. We believe that instead of showing all actions for users, showing only relevant actions and compacting the total buttons into four could reduce interface clutter.

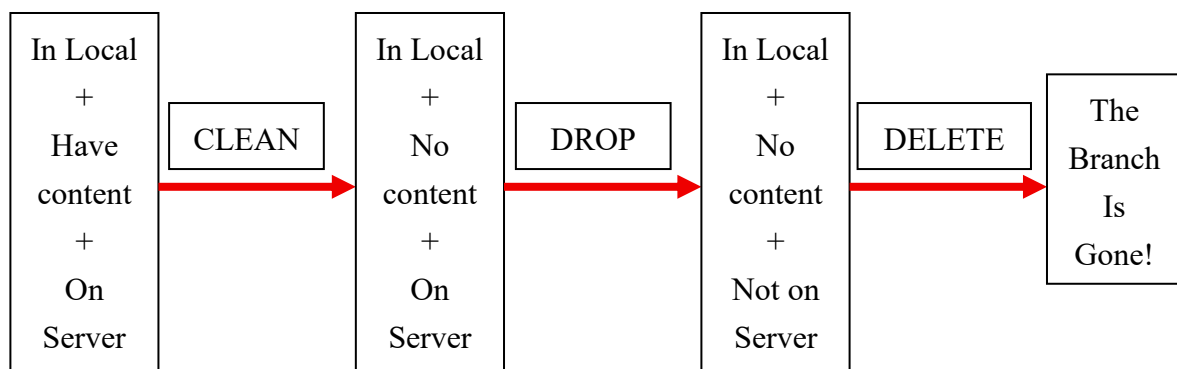
Button 1 – CHECKOUT/ SWITCH

The button label changes based on whether the selected branch is currently in [HEAD] status (activated branch):

Selected Branch State	Button Label	Action
Selected branch is HEAD	CHECKOUT	Download the branch data from the server and replace the local branch
Selected branch is not HEAD and is local	SWITCH	Change the active branch to the selected one
Selected branch is not HEAD and is not local	SWITCH	Download the branch data from the server and then switches to it

Button 2 – CLEAN/ DROP/ DELETE

This is a state machine driven operation button – each operation changes the status of the branch, and the status of a branch changes the operation behaviour:



The Authentication Page

Here are the guest status and logged in status of the authentication panel:

ACCOUNT AUTHENTICATION	ACCOUNT AUTHENTICATION
UID 	WELCOME test2
PASSCODE 	LOGOUT
LOGIN	EMAIL
REGISTER	REGISTER
RESET PASSCODE	INSTALL APP

Walkie-Typie Pages and Broadcast Pages

The operation logic is similar, because they are the extended application from Blackboard. If you understand single player video game culture, then you might analyse them as “DLCs”. We will briefly walk through the common use cases.

Here is the overview of their UI:

Walkie-Typie Controller	Broadcast Controller
WALKIE-TYPIE CONTROLLER test1 2026-02-24T15:32:03.160+08:00 test1 UID ADD CUT	CHANNEL MODULATION SEARCH CHANNELS... A Test Broadcast 2026-03-03T06:53:02.518+08:00 test title PIN CAST CREATE DELETE
Walkie-Typie Texting Page	Broadcast Text Area
BLACKBOARD WALKIE-TYPIE BROADCAST MODS LINK TEXT CONFIG PUSH NAKE NAME - TEST0 FULL PUSH TEST2 FULL ↕	BLACKBOARD WALKIE-TYPIE BROADCAST MODS A Test Broadcast CHANNE PUSH Broadcast Hello World01 📄 ☰ FULL

Walkie-Typie

It is a real-time one-to-one text communication feature; only registered users can connect with each other. Here is the operation flow:

1. User A and User B are logged in
2. User A enters User B's UID and presses ADD to build a connection
3. Once connected, both sides can see each other in the connection list
4. User A goes to the TEXT page and starts typing in the below-board area; user B sees the typing keystrokes streaming on the above-board area in real-time

Hint 1: The selected UID on the list will be automatically refreshed in the TEXT page

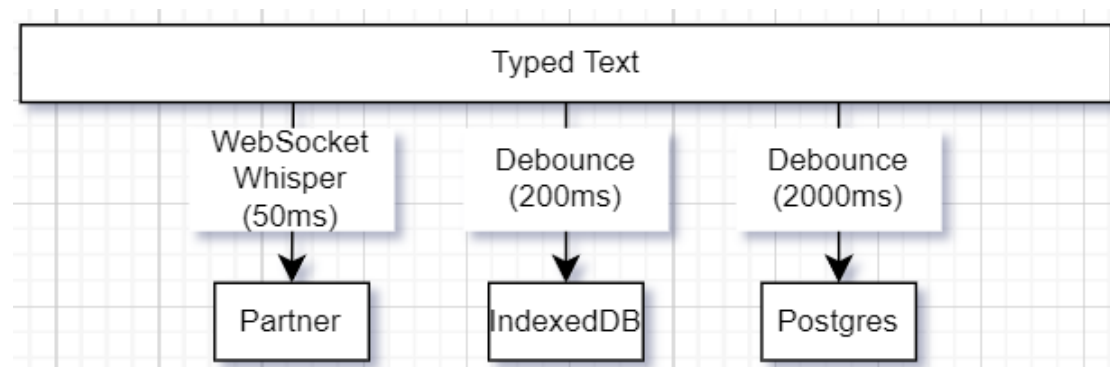
Hint 2: By default, below is your board, and the above one is others'

Hint 3: Users can swap the position by clicking the switch button

Hint 4: Although PUSH and PULL buttons do the navigation the same as Blackboard, when the other side is typing, the viewing page will be redirected to the page where the opponent is editing

Hint 5: To disconnect a user, select him and press CUT button

The data flow behind the scenes:



Although it requires Internet, it adopts local-first characteristics to enhance performance and reliability. When a user refreshes the connect object, the client first fetches both sides' committed history from the server via HTTP, but the self-owned board is persisted into IndexedDB (disk), while partner's one is in the memory (RAM). As a result, our own data survives connection-drop accidents (our local data has higher priority than server ones), and real-time sync to the other side is handled instantly via WebSocket Whisper.

Broadcast

This is an overseer-oriented system, which allows the UID granted with a title (which can only be granted from the backend (pgAdmin 4)) of the website to easily announce news publicly.

	id [PK] bigint	uid character varying (255)	passcode character varying (255)	title character varying (255)	email character varying (255)	remember_token character varying (100)	created_at timestamp without time zone	updated_at timestamp without time zone	settings jsonb
1	1	test1	\$2y\$12\$Z0vROYNF...	[null]	[null]	[null]	2026-02-24 07:24:52	2026-02-24 07:24:52	[null]
2	2	test2	\$2y\$12\$2MwXtou2E...	[null]	[null]	[null]	2026-02-24 07:30:53	2026-03-01 12:25:42	{'glo...
3	3	test0	\$2y\$12\$POWhsdIV5...	test title	[null]	[null]	2026-02-24 07:34:00	2026-02-24 07:34:00	[null]

This is the difference between a titled and untitled UID row

Titled HUD UID: test0 [test title] DB: ONLINE	Untitled HUD UID: test2 DB: ONLINE
--	---------------------------------------

The Mod System

MOD REGISTRY

v2.0.0

Answer the question below:
v4.1.0

— AVAILABLE —

ASCII ANIMATOR

ADDED

FILE ATTACH

ADD

INFO SCREENSAVER

ADD

LIGHT THEME

ADDED

LLM

ADD

UP

DOWN

DELETE

REFRESH

Step 1: Add a mod “LLM”. Then **prompt** in the config page as shown

Step 2: Click the button added by mod

Step 3: See the **result** of the conclusion to the **text area**

“Mod” is a video game term. It has different names in different domains, such as extension and add-on. Users are supposed to be able to customize their experience by different combinations of mods. We have made 10+ official mods for users to try by themselves; however, it is impossible to demonstrate all in this report, therefore, we will demonstrate only the LLM mod. Nevertheless, each mod has its own introduction and configuration.

SAVED: master: Explain the difference between TCP and UDP. Give one real-world application for each and explain why that protocol is more suitable than the other for that use case.

SAVED: master: Exp. UDP for is r that

Answer the question below:

****TCP (Transmission Control Protocol)**** is used for reliable data transfer, such as in real-time applications like online gaming or video streaming, where data must be guaranteed to reach the destination. It ensures that data is delivered without errors and with a guaranteed delay.

****UDP (User Datagram Protocol)**** is used for fast, short data transfer, such as in real-time applications like online gaming or video streaming. It is more suitable here because it offers lower latency and is often used

Each Mod is an Instance

Each mod works differently; some can be instantiated multiple times as a button; some can only be instantiated once and represented in some UI/UX features. Take the LLM mod as an example, it can be instantiated multiple times, each instantiation can have different prompts for different purposes.

Lazy Loading

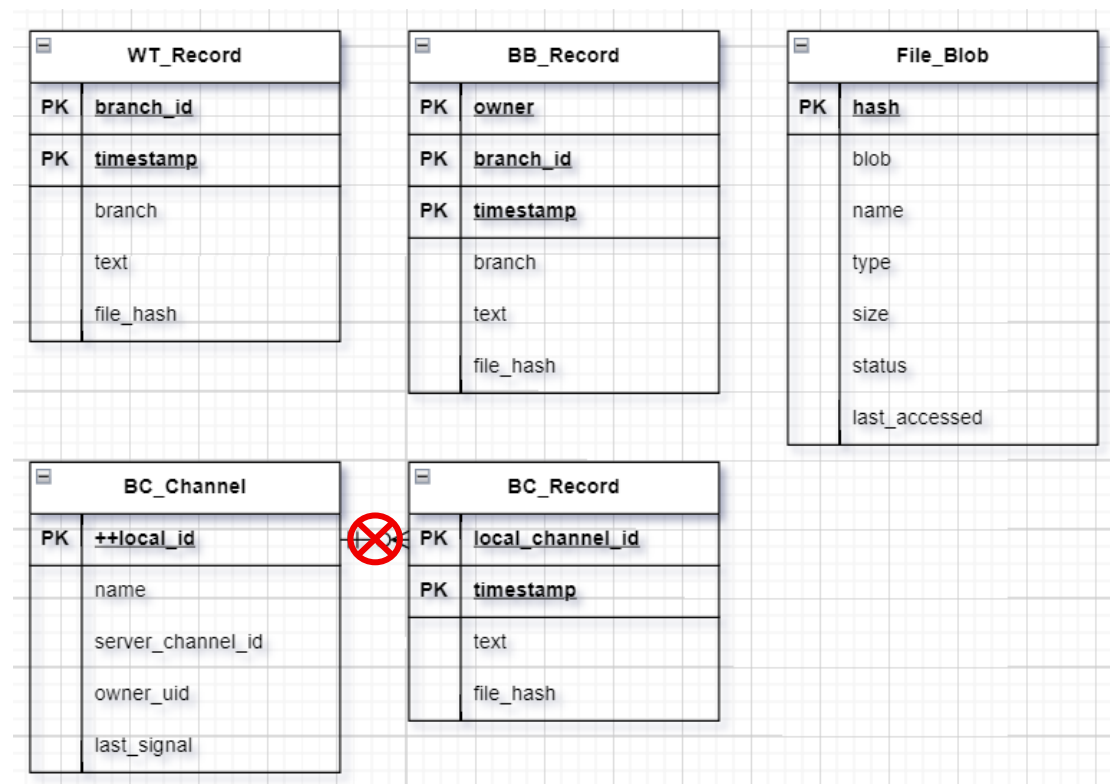
Phase 0: When the webpage loads, the system will first fetch all manifest files which describe each of the mods' metadata.

Phase 1: Only the mods that have been activated will be imported to the system and be executed.

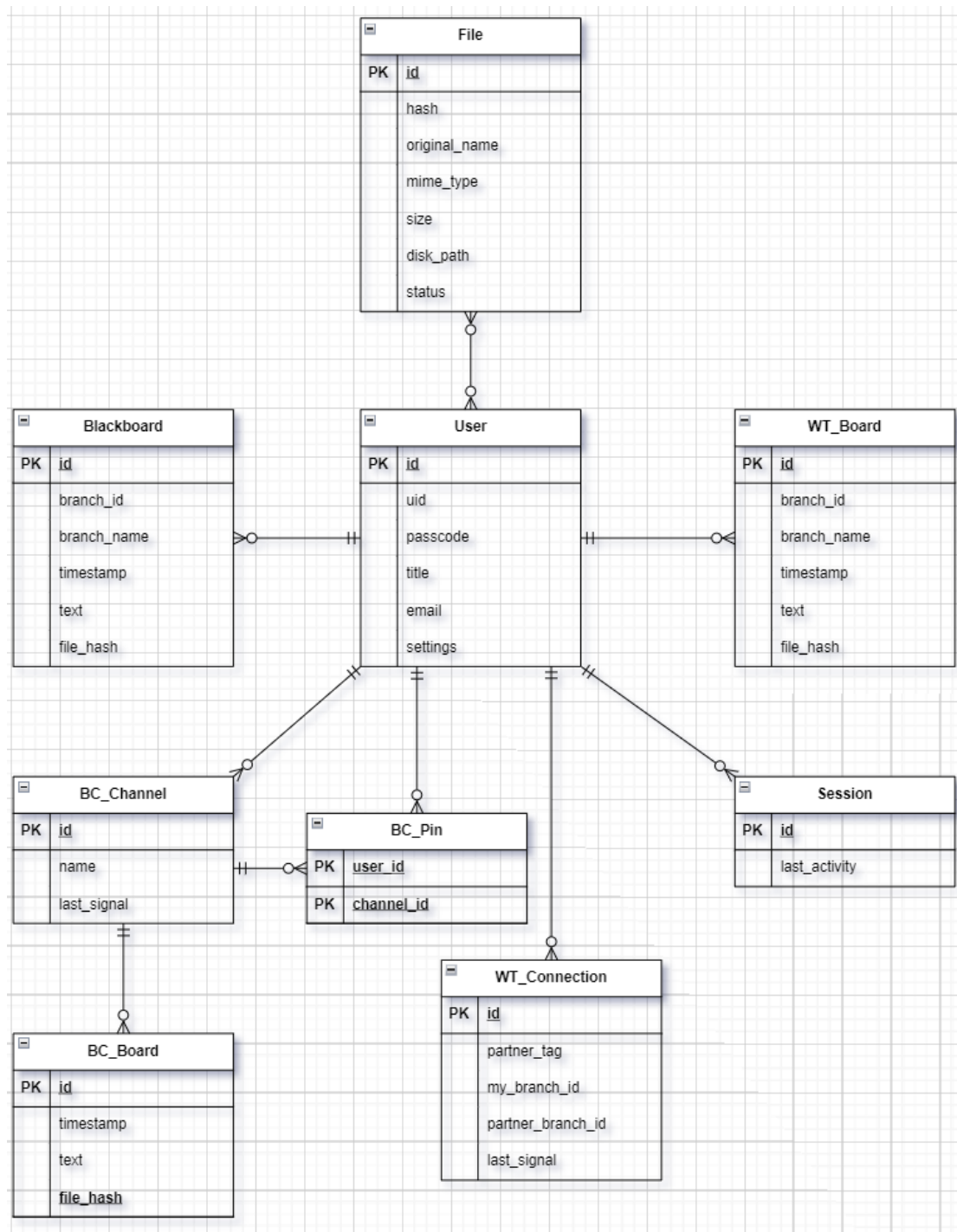
Entity Relationship Diagram (ERD)

Hint: IndexedDB is a NoSQL client database; therefore, no relationship lines are expected. Henceforth, we need to do the deletion for the channel manually (by code).

IndexedDB ERD



PostgreSQL ERD



Test Plan

System Test

Only focus on Blackboard due to page limitation

Step	Action	Expectation
1	Access https://my-cli.uk/	See a blank text area
2	Enter "B1-1" Then press shift + F5 to hard refresh	See if "B1-1" remains Prove IndexedDB on work
3	Enter "B1-2" to "B1-10" on each page After entering one, press push button Then keep pressing pull until last one to.	See if all records are on the correct index Prove IndexedDB on work
4	Goes to the page containing "B1-10" Then press push button Then enter "B1-11" Then pull until reach the last record	See if the record only have "B1-2" to "B1-11". Prove that auto cleaning by threshold on work
5	Edit the page containing "B1-6", change it to "B1-66"	See if the index of the modified page goes to the top Prove the timestamp update on work
6	Delete the page content containing "B1-66" Then press push button Then press pull button	See if the page is gone. Prove that blank-page-auto-clean feature on work
7	Create new branch by fork button Name it "B2"	See if branch creation features on work
8	Switch to branch "B2"	See if branch "B2" content are the same as original one
9	Register an account uid = test0 passcode = test0	Prove that auth and database on work
10	Login the account just created	Prove that auth and database on work
11	Press commit on branch "B2"	See if the branch status indicator updated
12	Go to new device and repeat step 1 and step 10	See if the branch list has "B2" Prove that the sync feature on work
13	Press switch/checkout on branch "B2"	See if the branch data is transmitted to the new device Prove that the sync feature on work
14	Press clean button	See if branch content of "B2" is wiped
15	Press drop button	See if the branch status of "B2" shows no longer on the database
16	Press delete button	See if the branch "B2" is deleted

Component Test (Unit/Integration Test)

#	Target	Input	HTTP Status Code
1	Register	{uid: hello world, passcode: abcd}	422
2	Register	{uid: hello world, passcode: abcd}	201
3	Register	{uid: hello world, passcode: abc}	422
4	Register	Registered uid	422
5	Login	Incorrect uid/ password	401
6	Login	Correct uid/password	200

#	Target	Action/Condition	Result
1	Any Board	Attach same file twice	Second attachment returns existing record

Acceptance Test

#	Test	Expectation
1	UI layout acceptance	User knows how to operate with navigation bar
2	UI theme acceptance	User accepts either default theme (dark mode) or light mode from mod options
3	Blackboard operation	User accepts push and pull operation and concept
4	Blackboard branch operation	User understands the operation of four buttons inside of branch controller by reading hints on hint window (by hovering a button, a hint will show up)
5	Walkie-Typie chat	User connects with others and starts chatting without guideline
6	Broadcast browsing	User realizes the concept of the Broadcast is like notice board

System Architecture

List of Architecture of System

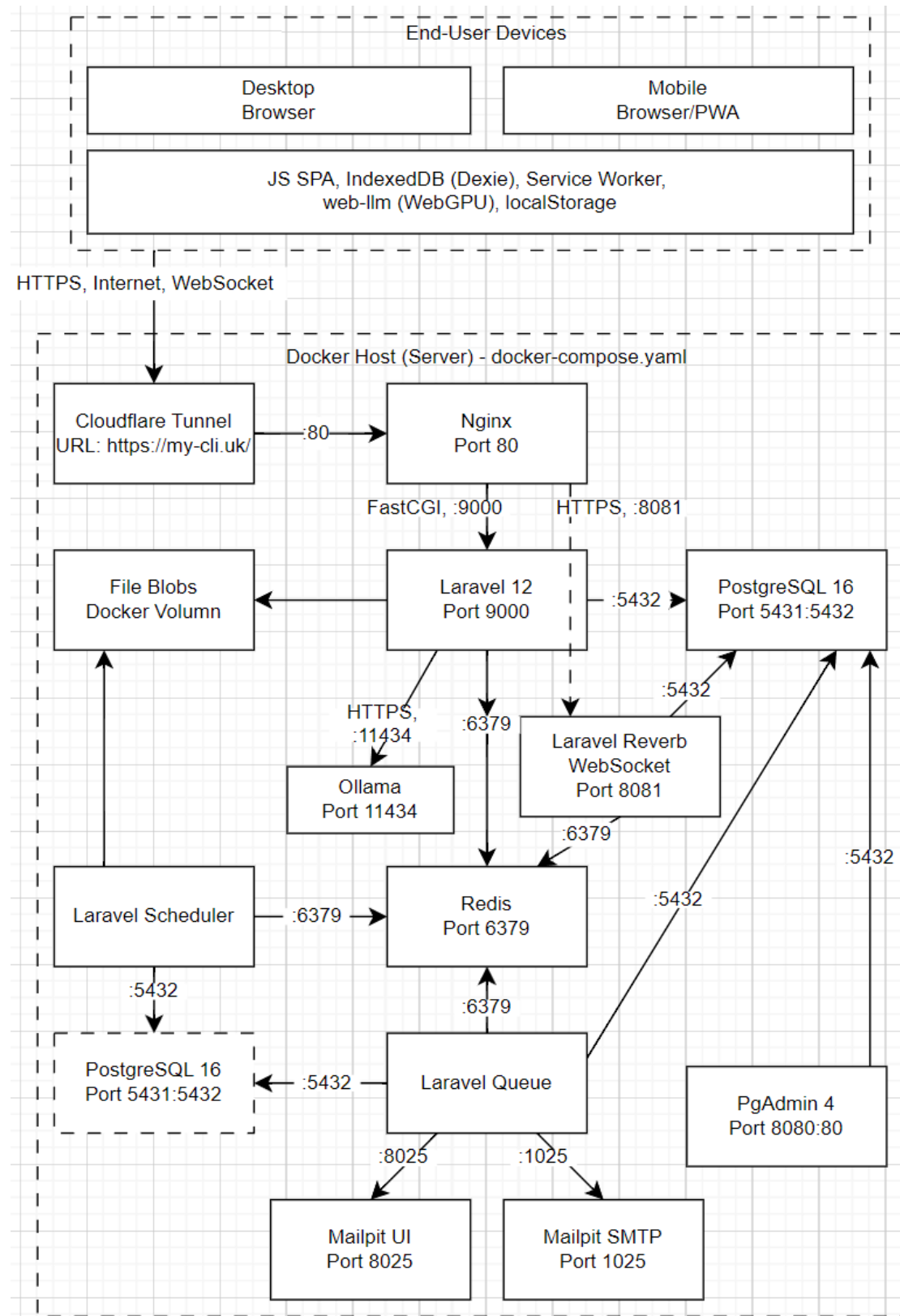
Server (Docker)

Technology/Methodology/API	Role
Nginx Alpine	Gateway
PHP	Runtime
Laravel 12	Application framework
Laravel Queue	Async job worker
Laravel Scheduler	Cleaning orphan data
Laravel Reverb	WebSocket server
WebSocket	Real-time transmission
PostgreSQL 16	Relational server database
Redis Alpine	Caching, such that allow shorter request debounce
Ollama	Server LLM host for Qwen-3.5:4b
Cloudflare Tunnel	Network tunneling for home server
pgAdmin 4	Database UI
Mailpit	Email send/receive test

Client

Technology/Methodology/API	Role
Vanilla HTML	Structuring; Single Page Application (SPA)
Vanilla CSS	Styling
Vanilla JS	Logic
localStorage	Config storage
IndexedDB	Local database
Dexie.js	IndexedDB wrapper library
Service Worker	Offline caching
Workbox	Build Service Worker
i18n	Internationalization system with three text presentation
Makred.js	Markdown rendering for mod
Laravel Echo IIFE	Client WebSocket
Pusher.js	Laravel Echo IIFE prerequisite
web-llm	Client LLM using WebGPU for Qwen-3.0 series
Textmode.js	WebGL for ASCII art rendering mod
Google Cloud API	Translation and Speech-to-Text
smtp.google.com	Email delivery for email registration and password reset

Architectural Diagram



Cost Estimation

Minimal Deployment Fee

Hardware/Software/Service	Estimated Cost (HKD)
A computer with RTX 4080 as server	\$13000
Cloudflare Tunnel	\$0
Domain Name	\$40/year
Google Cloud Translation API	\$0 (<500K character per month)
Google Cloud Speech-to-Text API	\$0.13/minute
Google SMTP Service	\$0

Minimum Viable Product Cost

Manpower	Man-Day	Rate (HKD/day)	Cost (HKD)
Solo Developer	30	\$0	\$0

Tool	Cost
Claude Max 5x	\$780/month

Backup Strategy

GitHub repository as backup. Me, as a developer, can resume if the source code is polluted.

You may visit the repository by “https://github.com/HKPooPoo/My_CLI_Re”

Any developer can execute this command to pull the repository:

“git clone https://github.com/HKPooPoo/My_CLI_Re.git”

Conclusion

This report presents the overall design of MyCLI. However, the capacity of the report does not allow us to explain each component in detail.

By adopting various insights from video games, we hope that different people can use this software according to their needs. Although the three core - Blackboard, Walkie-Typie, and Broadcast have fundamentally extended the application scenario, the mod system provides it more potential for further development.

Hint: You may access the website by “<https://my-cli.uk/>”. However, the project is work in process, the server (home computer) might shutdown due to personal needs, therefore, it might be unstable due to software version and cache problem.

Work Distribution

Student Name	Work
YU Shing Hei	All sections: Proposal, System Analysis, System Design, UI/UX Design, Database Design, Test Plan, Architecture Design, Frontend Development, Backend Development, Deployment, Report Writing

Reference

- Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2>
- Cloudflare. (n.d.). Cloudflare Tunnel documentation. <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>
- Docker. (n.d.). Docker documentation. <https://docs.docker.com/>
- Fahlander, D. (n.d.). Dexie.js documentation. Dexie.js. <https://dexie.org/docs/>
- Google Cloud. (n.d.). Cloud Speech-to-Text API documentation. <https://cloud.google.com/speech-to-text/docs>
- Google Cloud. (n.d.). Cloud Translation API documentation. <https://cloud.google.com/translate/docs>
- Google. (n.d.). Workbox: Production-ready service worker libraries. Chrome for Developers. <https://developer.chrome.com/docs/workbox>
- humanbydefinition. (n.d.). textmode.js [Computer software]. GitHub. <https://github.com/humanbydefinition/textmode.js>
- Laravel. (n.d.). Laravel 12.x documentation. <https://laravel.com/docs/12.x>
- Mailpit. (n.d.). Mailpit [Computer software]. GitHub. <https://github.com/axllent/mailpit>
- Marked. (n.d.). Marked: A markdown parser and compiler. <https://marked.js.org/>
- MLC-AI. (n.d.). WebLLM [Computer software]. GitHub. <https://github.com/mlc-ai/web-llm>
- Mozilla. (n.d.). MDN Web Docs. <https://developer.mozilla.org/>
- Nginx. (n.d.). Nginx documentation. <https://nginx.org/en/docs/>
- Ollama. (n.d.). Ollama [Computer software]. GitHub. <https://github.com/ollama/ollama>
- pgAdmin Development Team. (n.d.). pgAdmin 4 documentation. <https://www.pgadmin.org/docs/>
- The PHP Group. (n.d.). PHP documentation. <https://www.php.net/docs.php>
- PostgreSQL Global Development Group. (n.d.). PostgreSQL 16 documentation. <https://www.postgresql.org/docs/16/>
- Pusher. (n.d.). Pusher Channels documentation. <https://pusher.com/docs/channels/>
- Redis Ltd. (n.d.). Redis documentation. <https://redis.io/docs/>